Narrative and Debugging Overview of Cellphone Data Cleaning Functions

David Ewing on behalf of DATA422-LAB 02 GROUP E

2024.10.10

Abstract

This document provides an explanation of the cleaning functions implemented for processing cellphone data files. The data consists of information from two vendors, each containing columns for time, SA2 codes, and the number of calls per hour over two weeks. The goal of these functions is to clean, process, and validate the data by removing duplicates, managing missing or zero values, and maintaining data integrity throughout processing. Furthermore, the document also covers the sequence of function calls and their specific debugging details.

1 Introduction

The data cleaning functions in this document are part of a broader data processing pipeline designed to handle cellphone data from two vendors. The raw data contains issues such as missing values (NA), zero counts, and duplicate entries. Cleaning these issues is critical for reliable analysis and further usage. The functions are organised and called in sequence as per the overarching script, ensuring data consistency and reproducibility. The functions are all contained in the file `002_clean_celldata.R`, while the overarching script that guides their execution is `test-cellphone-functions.R`.

2 Function Overview

The functions in `002_clean_celldata.R` are primarily designed for preprocessing cellphone data from different vendors. The following is a description of each function, along with its role in the data cleaning process.

2.1 add_NA0

This function, `add_NA0(tb)`, adds two new columns to the input dataframe: `has_NA` and `has_00`. The column `has_NA` marks rows that contain `NA` in the `count` column, while the column `has_00` identifies rows where the `count` value is zero. Both columns take values of either `1` or `0`, representing whether the condition is met.

2.2 remove_NA0

The `remove_NA0(tb)` function removes the `has_NA` and `has_00` columns from the dataframe. This function is usually called after data with missing or zero values has been appropriately handled.

2.3 remove_istimed

This function, `remove_istimed(tb)`, removes the `is_after_start` and `is_before_end` columns that are used to validate whether the `datetime` values fall within a specified range.

2.4 preprocess_vf_data and preprocess_sp_data

These functions are designed to preprocess the raw data from the two vendors:
`preprocess_vf_data(tb)`:
- Renames `area` to `sa2` and converts it to numeric. - Renames `dt` to `datetime` and `devices` to `count`. - Calls `add_NA0` to add the `has_NA` and `has_00` columns and filters out rows with NA or zero values.
`preprocess_sp_data(tb)`:
- Similar to `preprocess_vf_data`, but operates on `ts` and `cnt` columns.

2.5 time_bound

The `time_bound(tb, time_start, time_end, tb_name)` function is responsible for ensuring that the data only includes entries within a specific time range. The `datetime` column is also converted to the New Zealand Standard Time (NZST) timezone.

2.6 merge_processed_celldata

This function merges the cleaned data from both vendors. It uses a full join to ensure both datasets are properly merged on `NZST` and `sa2` and filters out rows with incomplete data.

2.7 sum_unique_duplicates

This function, `sum_unique_duplicates(tb)`, groups data by `NZST` and `sa2`, retains only unique `count` values, and sums them.

2.8 has_duplicates and process_duplicates

The `has_duplicates(tb)` function checks whether the dataframe has any duplicates based on `NZST` and `sa2`. If any duplicates are found, they are printed for analysis.
The `process_duplicates(tb)` function handles the duplicates by grouping and retaining unique values and summing them.

3 Debugging Process for Clean Functions

The debugging details of the functions in `002_clean_celldata.R` are essential to understanding how issues in data processing were addressed.

3.1 Debugging Details

- `add_NA0`: During the execution of `add_NA0`, debugging is facilitated by printing information using `cat()` statements. This helps ensure that the columns `has_NA` and `has_00` are accurately added. A `browser()` call is included to provide an interactive debugging session.

- `remove_NA0` and `remove_istimed`: These functions are relatively straightforward, but `cat()` statements are included to confirm their operation, especially for verifying column removals.

- `preprocess_vf_data and preprocess_sp_data`: Debugging includes checking that renaming and data type conversions were successful, as well as ensuring that rows with `NA` and zero counts were filtered out correctly.

- `time_bound`: Debugging focuses on ensuring the `datetime` column is properly converted to NZST. Checks are also included to verify the correct application of the time bounds.

- `merge_processed_celldata`: The debugging statements are used to monitor the join operation and validate that only complete records are retained. The `browser()` call provides an interactive environment to explore issues that arise during the join.
- `sum_unique_duplicates`: The function is debugged by first printing the intermediate dataframe. This helps verify that unique values are retained before summing.
- `has_duplicates and process_duplicates`: These functions include extensive debugging statements. The `process_duplicates` function has a mechanism to print only the first five sets of duplicates for detailed analysis, which is crucial for ensuring that the deduplication process is effective.

4 Conclusion

The functions in `002_clean_celldata.R` provide a robust mechanism for preprocessing cellphone data from multiple vendors. By dealing effectively with `NA` values, zero counts, and duplicates, the data is prepared for reliable analysis. Debugging plays a vital role in confirming the accuracy of each operation, ensuring that data integrity is maintained throughout the cleaning process.